UANL
UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FIME
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Nombres	**Matriculas**
Josue Yahir Vela Hernandez	2142881
Osweld Omar Lozano Martínez	2142366
Sylvia Yadira González Parra	2142962
Nombre del curso:	**Nombre del profesor:**
Componentes de lógica de negocio	Ing. Miguel Ángel Gómez Marroquin
Actividad:	**Fecha:**
Unidad 4	28/10/2026

Contenido

1. README.md con pasos de instalación, ejecución, reglas y endpoints. 2

2. Captura de carga exitosa de los 16 pedidos. 4

3. Bitácora resuelta de los 15 escenarios E01–E15. 5

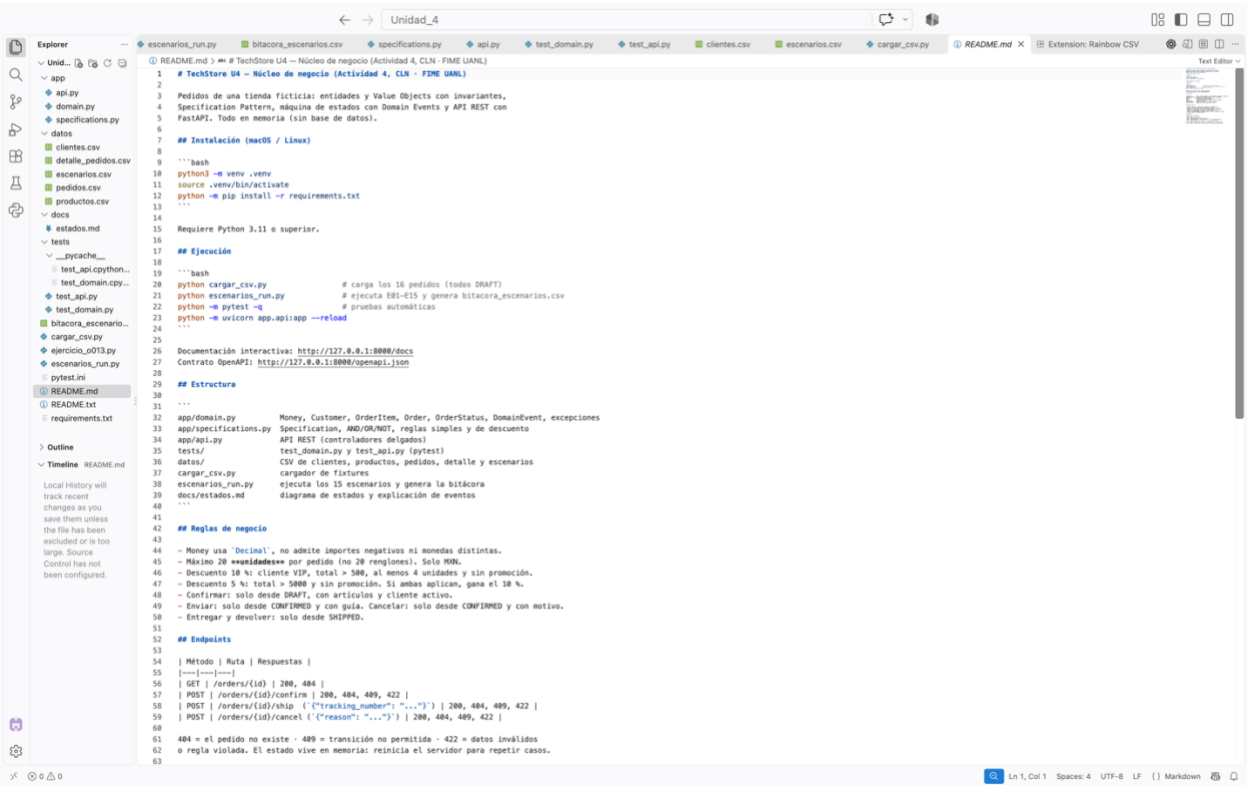
4. Pruebas pytest y captura de resultados. 6

5. Capturas de Swagger /docs mostrando una operación exitosa y una rechazada. 7

6. Diagrama de estados y explicación de los Domain Events. 9

7. Reflexión breve: tres errores encontrados, cómo se solucionaron y qué aprendizaje se obtuvo. 10

1. **README.md con pasos de instalación, ejecución, reglas y endpoints.**



```
# TechStore U4 — Núcleo de negocio (Actividad 4, CLN · FIME UANL)

Pedidos de una tienda ficticia: entidades y Value Objects con invariantes,
Specification Pattern, máquina de estados con Domain Events y API REST con
FastAPI. Todo en memoria (sin base de datos).

## Instalación (macOS / Linux)

```bash
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Requiere Python 3.11 o superior.

Ejecución

```bash
python cargar_csv.py              # carga los 16 pedidos (todos DRAFT)
python escenarios_run.py          # ejecuta E01-E15 y genera bitacora_escenarios.csv
python -m pytest -q               # pruebas automáticas
python -m uvicorn app.api:app --reload
```

Documentación interactiva: http://127.0.0.1:8000/docs
Contrato OpenAPI: http://127.0.0.1:8000/openapi.json

Estructura

```
app/domain.py          Money, Customer, OrderItem, Order, OrderStatus, DomainEvent, excepciones
app/specifications.py  Specification, AND/OR/NOT, reglas simples y de descuento
app/api.py             API REST (controladores delgados)
tests/                 test_domain.py y test_api.py (pytest)
datos/                 CSV de clientes, productos, pedidos, detalle y escenarios
cargar_csv.py          cargador de fixtures
escenarios_run.py      ejecuta los 15 escenarios y genera la bitácora
docs/estados.md        diagrama de estados y explicación de eventos
```

Reglas de negocio

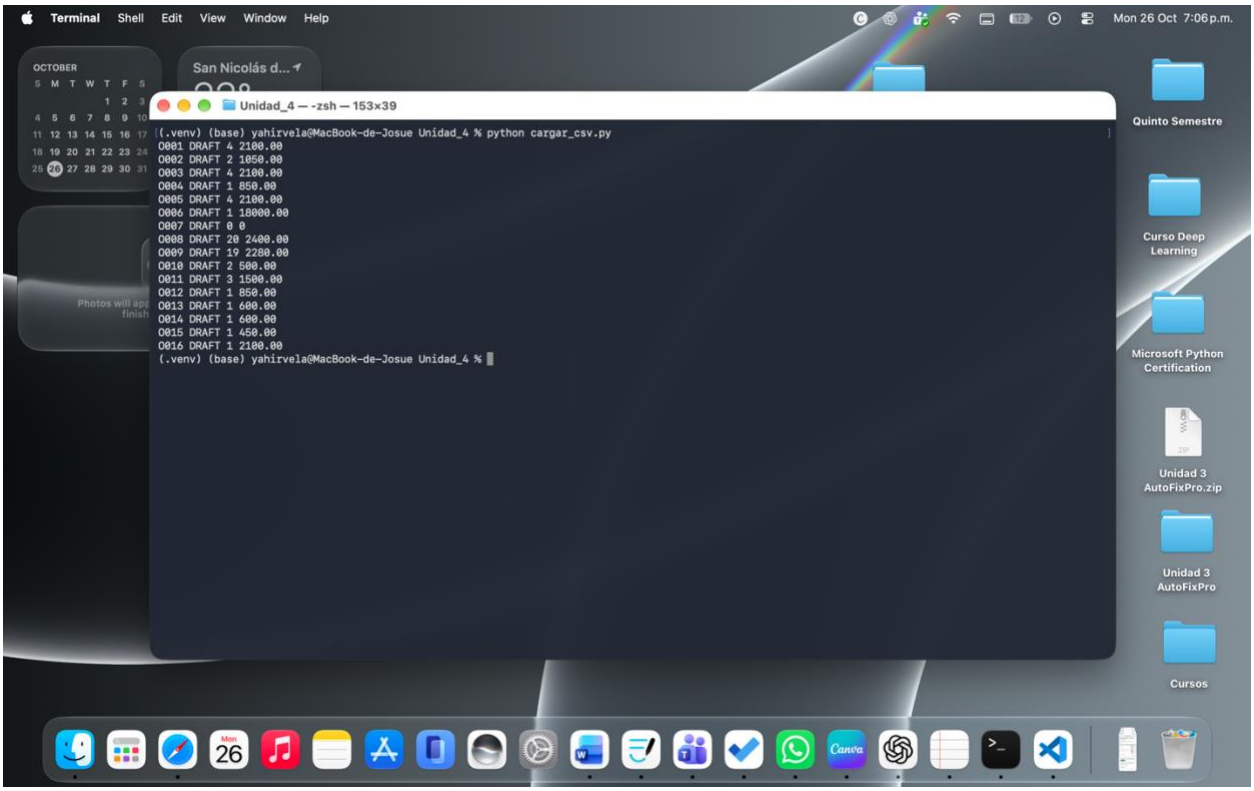
- Money usa `Decimal`, no admite importes negativos ni monedas distintas.
- Máximo 20 **unidades** por pedido (no 20 renglones). Solo MXN.
- Descuento 10 %: cliente VIP, total > 500, al menos 4 unidades y sin promoción.
- Descuento 5 %: total > 5000 y sin promoción. Si ambas aplican, gana el 10 %.
- Confirmar: solo desde DRAFT, con artículos y cliente activo.
- Enviar: solo desde CONFIRMED y con guía. Cancelar: solo desde CONFIRMED y con motivo.
- Entregar y devolver: solo desde SHIPPED.

Endpoints

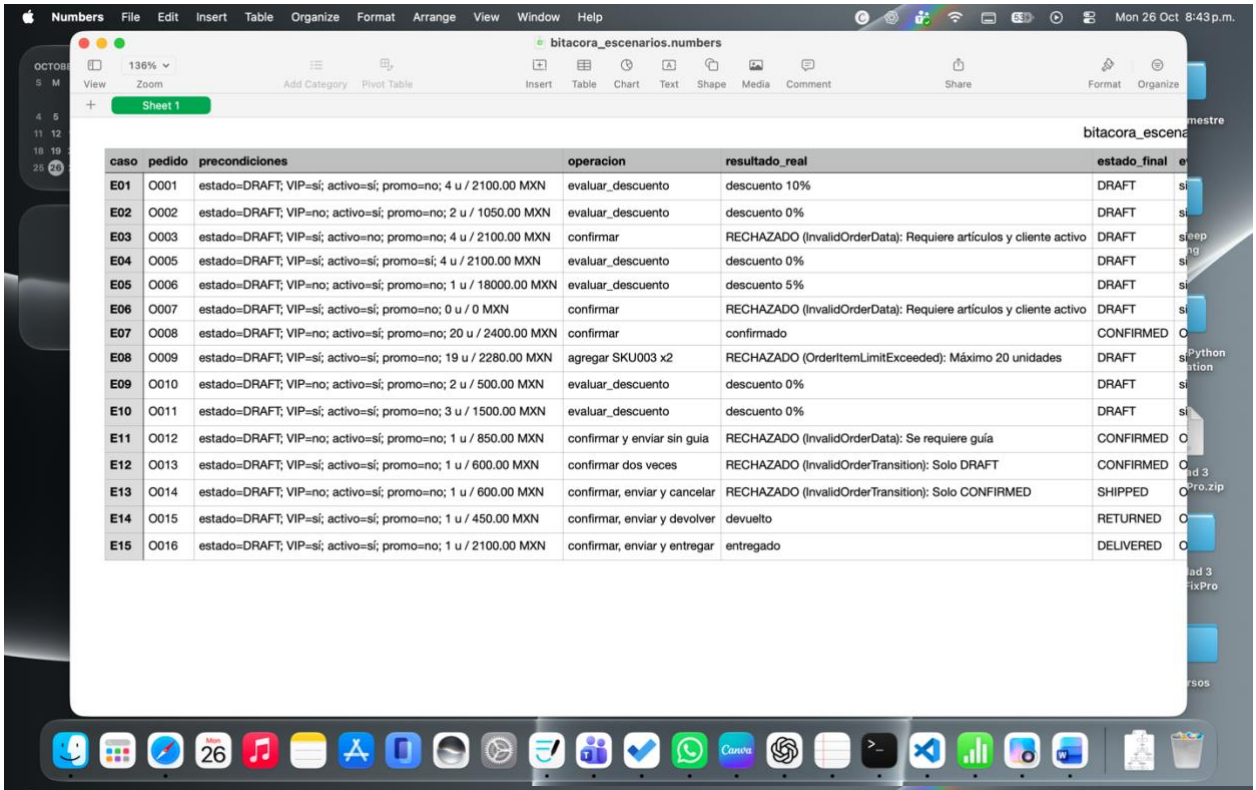
Método	Ruta	Respuestas
GET	/orders/{id}	200, 404
POST	/orders/{id}/confirm	200, 404, 409, 422
POST	/orders/{id}/ship (`{"tracking_number": "..."}`)	200, 404, 409, 422
POST	/orders/{id}/cancel (`{"reason": "..."}`)	200, 404, 409, 422

404 = el pedido no existe · 409 = transición no permitida · 422 = datos inválidos
o regla violada. El estado vive en memoria: reinicia el servidor para repetir casos.
```

2. Captura de carga exitosa de los 16 pedidos.

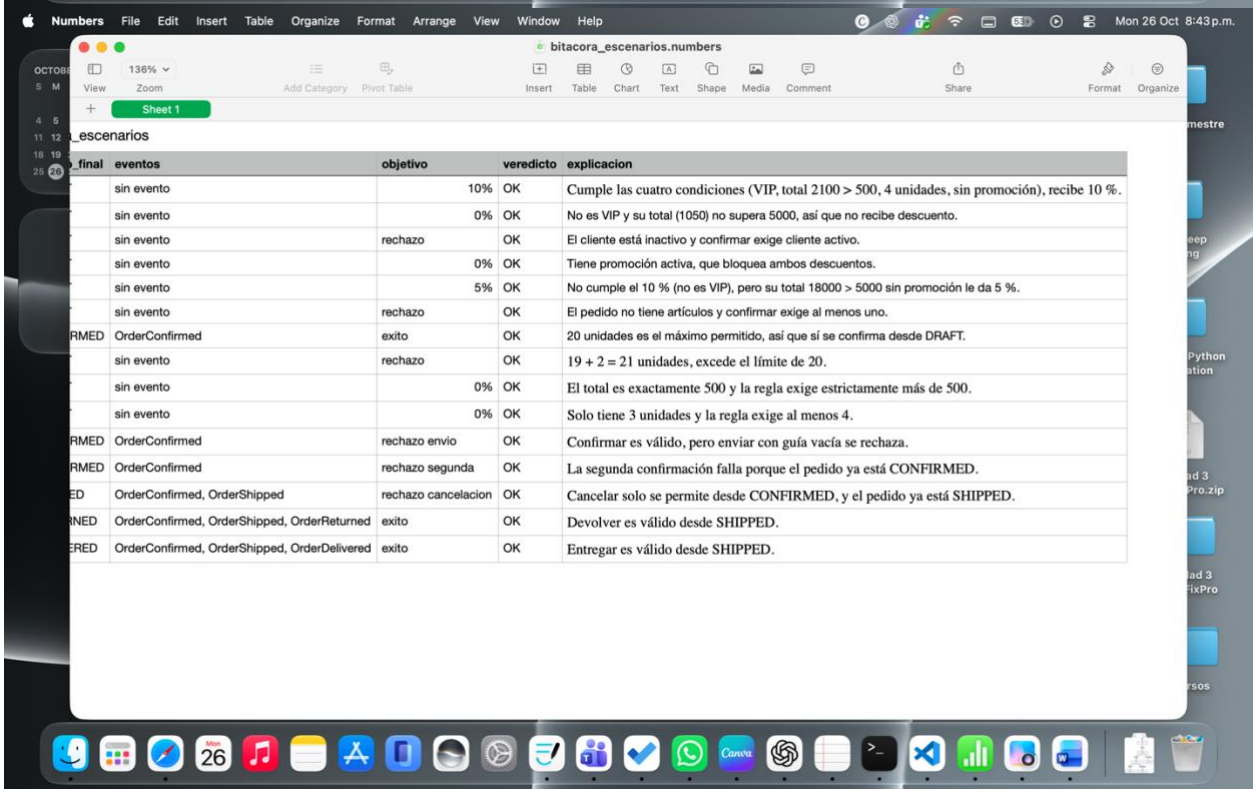


3. Bitácora resuelta de los 15 escenarios E01–E15.



bitacora_escena

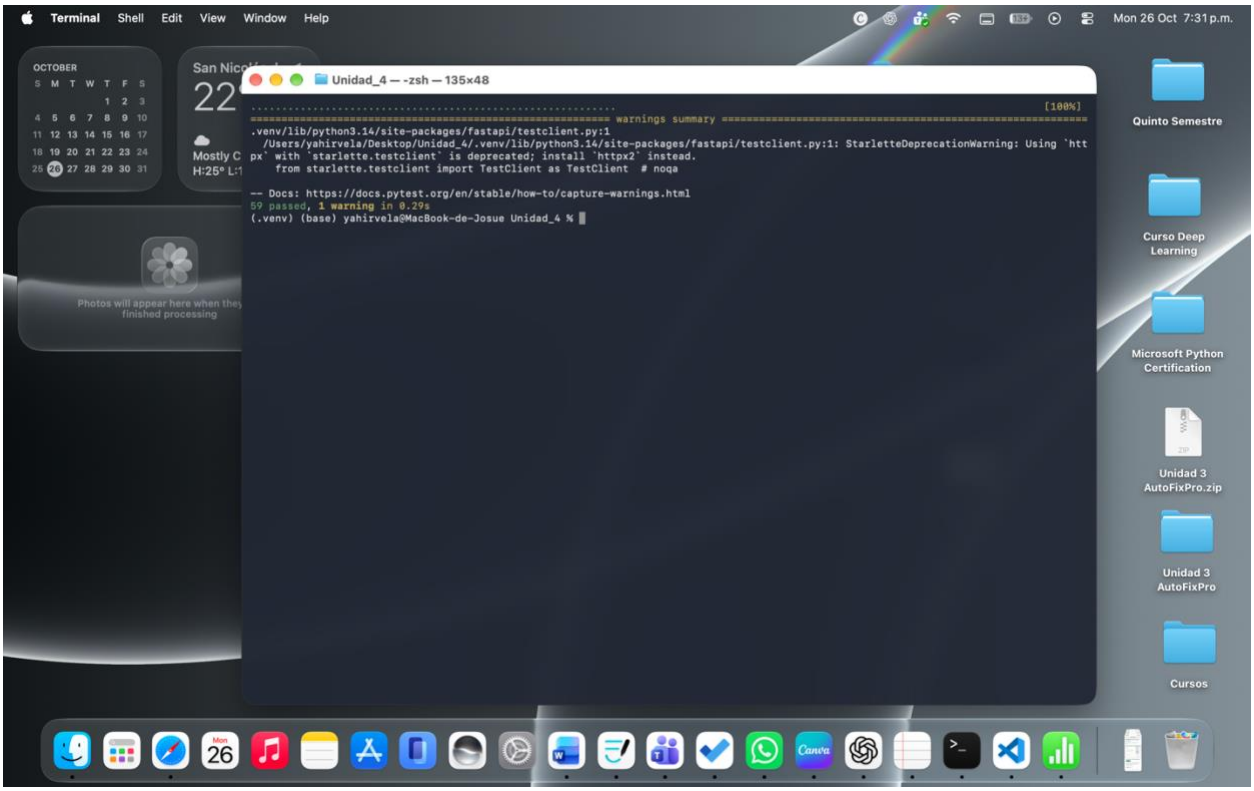
caso	pedido	precondiciones	operacion	resultado_real	estado_final	e
E01	O001	estado=DRAFT; VIP=sí; activo=sí; promo=no; 4 u / 2100.00 MXN	evaluar_descuento	descuento 10%	DRAFT	si
E02	O002	estado=DRAFT; VIP=no; activo=sí; promo=no; 2 u / 1050.00 MXN	evaluar_descuento	descuento 0%	DRAFT	si
E03	O003	estado=DRAFT; VIP=sí; activo=no; promo=no; 4 u / 2100.00 MXN	confirmar	RECHAZADO (InvalidOrderData): Requiere artículos y cliente activo	DRAFT	si
E04	O005	estado=DRAFT; VIP=sí; activo=sí; promo=sí; 4 u / 2100.00 MXN	evaluar_descuento	descuento 0%	DRAFT	si
E05	O006	estado=DRAFT; VIP=no; activo=sí; promo=no; 1 u / 18000.00 MXN	evaluar_descuento	descuento 5%	DRAFT	si
E06	O007	estado=DRAFT; VIP=sí; activo=sí; promo=no; 0 u / 0 MXN	confirmar	RECHAZADO (InvalidOrderData): Requiere artículos y cliente activo	DRAFT	si
E07	O008	estado=DRAFT; VIP=no; activo=sí; promo=no; 20 u / 2400.00 MXN	confirmar	confirmado	CONFIRMED	O
E08	O009	estado=DRAFT; VIP=sí; activo=sí; promo=no; 19 u / 2280.00 MXN	agregar SKU003 x2	RECHAZADO (OrderItemLimitExceeded): Máximo 20 unidades	DRAFT	si
E09	O010	estado=DRAFT; VIP=no; activo=sí; promo=no; 2 u / 500.00 MXN	evaluar_descuento	descuento 0%	DRAFT	si
E10	O011	estado=DRAFT; VIP=sí; activo=sí; promo=no; 3 u / 1500.00 MXN	evaluar_descuento	descuento 0%	DRAFT	si
E11	O012	estado=DRAFT; VIP=no; activo=sí; promo=no; 1 u / 850.00 MXN	confirmar y enviar sin guia	RECHAZADO (InvalidOrderData): Se requiere guía	CONFIRMED	O
E12	O013	estado=DRAFT; VIP=sí; activo=sí; promo=no; 1 u / 600.00 MXN	confirmar dos veces	RECHAZADO (InvalidOrderTransition): Solo DRAFT	CONFIRMED	O
E13	O014	estado=DRAFT; VIP=no; activo=sí; promo=no; 1 u / 600.00 MXN	confirmar, enviar y cancelar	RECHAZADO (InvalidOrderTransition): Solo CONFIRMED	SHIPPED	O
E14	O015	estado=DRAFT; VIP=sí; activo=sí; promo=no; 1 u / 450.00 MXN	confirmar, enviar y devolver	devuelto	RETURNED	O
E15	O016	estado=DRAFT; VIP=sí; activo=sí; promo=no; 1 u / 2100.00 MXN	confirmar, enviar y entregar	entregado	DELIVERED	O



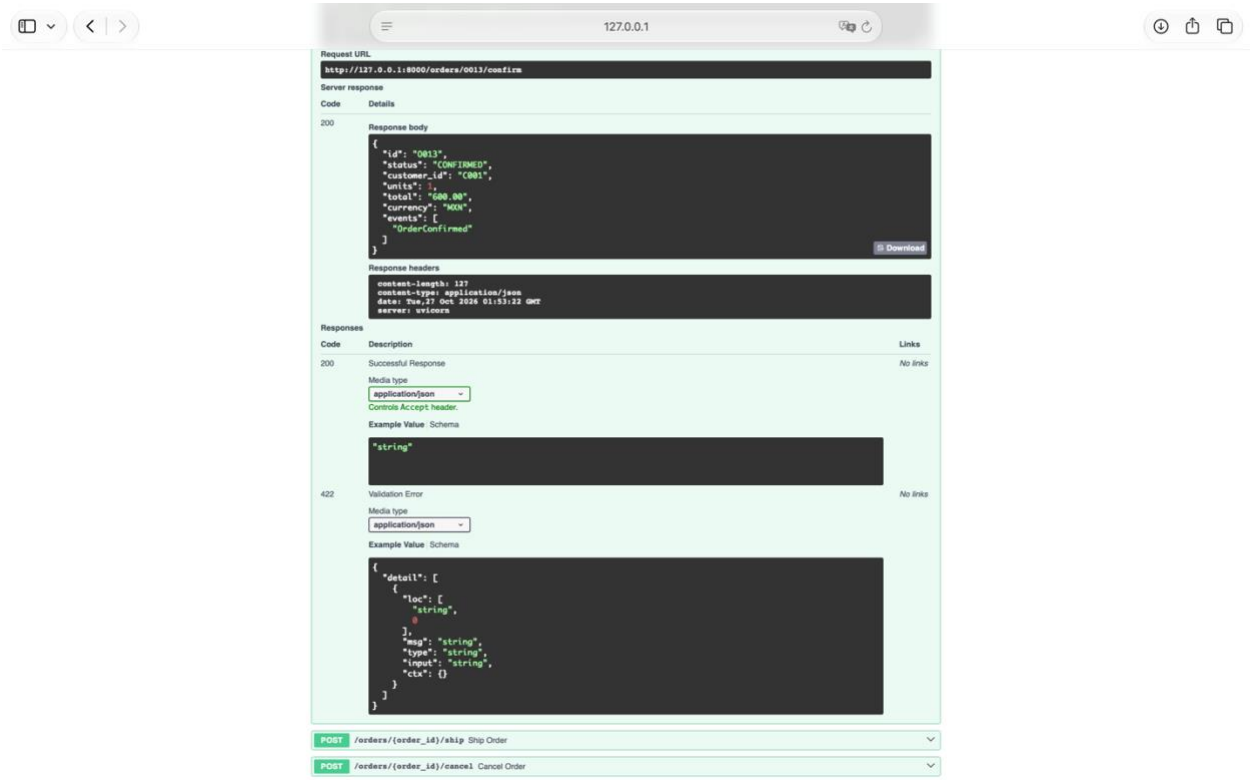
_escenarios

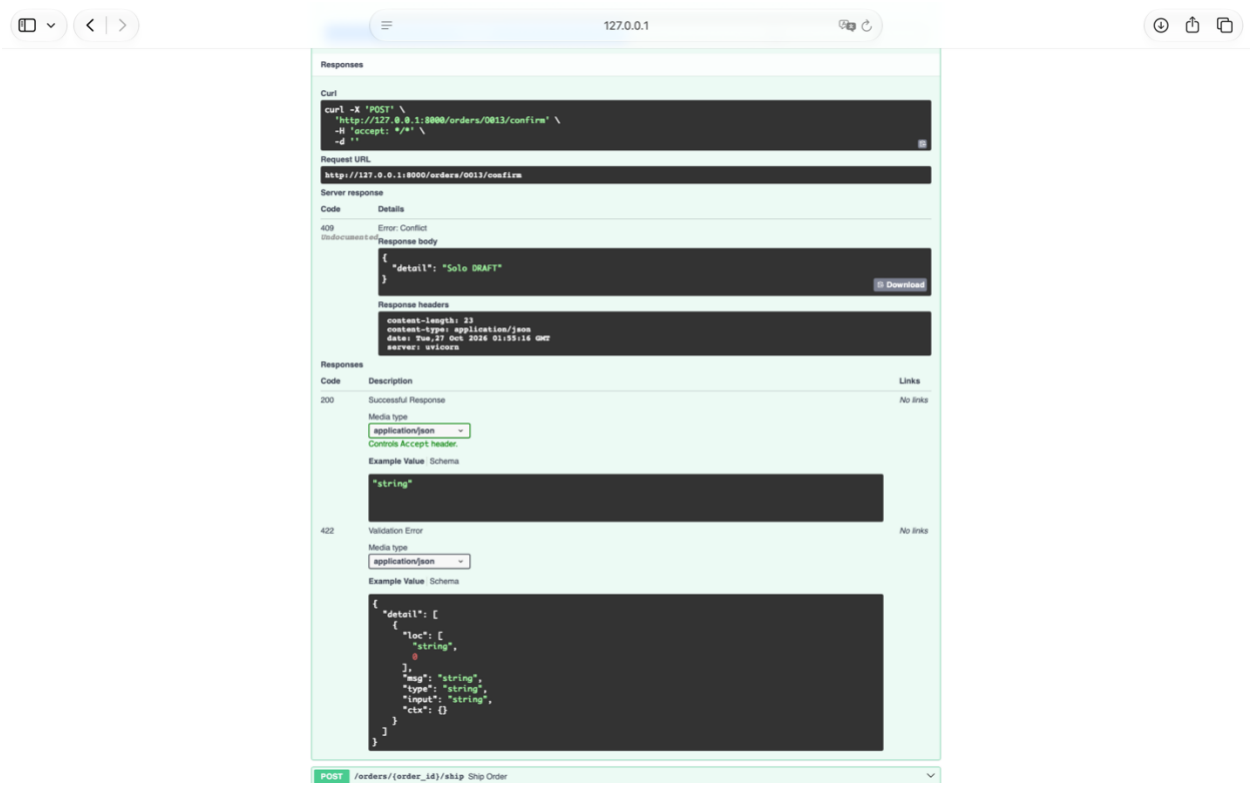
_final	eventos	objetivo	veredicto	explicacion
	sin evento	10%	OK	Cumple las cuatro condiciones (VIP, total 2100 > 500, 4 unidades, sin promoción), recibe 10 %.
	sin evento	0%	OK	No es VIP y su total (1050) no supera 5000, así que no recibe descuento.
	sin evento	rechazo	OK	El cliente está inactivo y confirmar exige cliente activo.
	sin evento	0%	OK	Tiene promoción activa, que bloquea ambos descuentos.
	sin evento	5%	OK	No cumple el 10 % (no es VIP), pero su total 18000 > 5000 sin promoción le da 5 %.
	sin evento	rechazo	OK	El pedido no tiene artículos y confirmar exige al menos uno.
RMED	OrderConfirmed	exito	OK	20 unidades es el máximo permitido, así que sí se confirma desde DRAFT.
	sin evento	rechazo	OK	19 + 2 = 21 unidades, excede el límite de 20.
	sin evento	0%	OK	El total es exactamente 500 y la regla exige estrictamente más de 500.
	sin evento	0%	OK	Solo tiene 3 unidades y la regla exige al menos 4.
RMED	OrderConfirmed	rechazo envio	OK	Confirmar es válido, pero enviar con guía vacía se rechaza.
RMED	OrderConfirmed	rechazo segunda	OK	La segunda confirmación falla porque el pedido ya está CONFIRMED.
ED	OrderConfirmed, OrderShipped	rechazo cancelacion	OK	Cancelar solo se permite desde CONFIRMED, y el pedido ya está SHIPPED.
NED	OrderConfirmed, OrderShipped, OrderReturned	exito	OK	Devolver es válido desde SHIPPED.
ERED	OrderConfirmed, OrderShipped, OrderDelivered	exito	OK	Entregar es válido desde SHIPPED.

4. Pruebas pytest y captura de resultados.

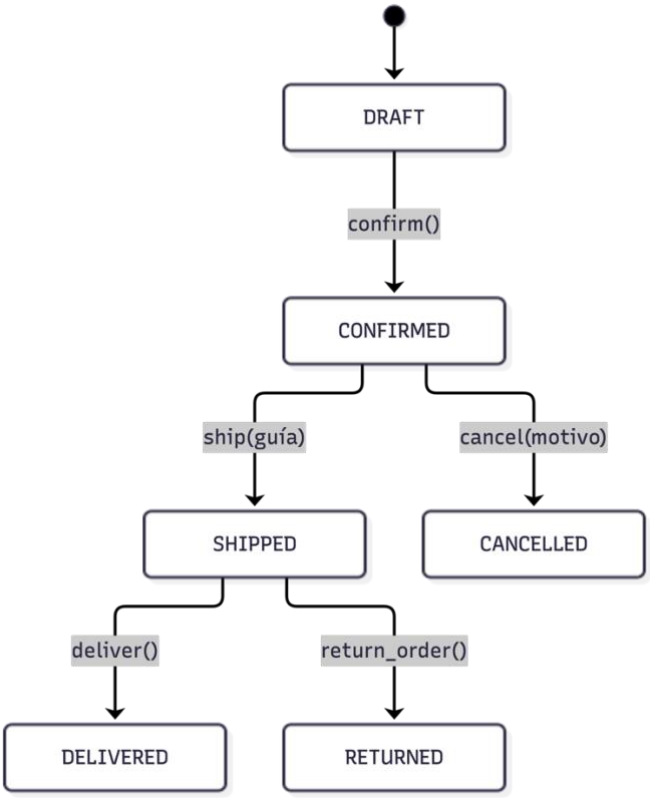


5. Capturas de Swagger /docs mostrando una operación exitosa y una rechazada.




127.0.0.1
Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8000/orders/0013/confirm' \
-H 'accept: */*' \
-d ''
Request URL
http://127.0.0.1:8000/orders/0013/confirm
Server response
Code
Details
409
Undocumented
Error: Conflict
Response body
{
"detail": "Solo DRAFT"
}
Download
Response headers
content-length: 23
content-type: application/json
date: Tue,27 Oct 2026 01:55:16 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value
Schema
"string"
422
Validation Error
No links
Media type
application/json
Example Value
Schema
{
"detail": [
{
"loc": [
"string",
0
],
"msg": "string",
"type": "string",
"input": "string",
"ctx": {}
}
]
}
POST
/orders/{order_id}/ship
Ship Order

6. Diagrama de estados y explicación de los Domain Events.



Un Domain Event es un registro inmutable de un hecho que ya ocurrió en el negocio. En este proyecto, Order registra un evento en cada transición
válida: OrderConfirmed, OrderShipped, OrderCancelled, OrderDelivered y OrderReturned. Cada método valida primero, cambia el estado después y registra el evento al final. Por eso una operación fallida, como confirmar dos veces, no deja ningún evento: no se anuncia un hecho que nunca pasó.

7. Reflexión breve: tres errores encontrados, cómo se solucionaron y qué aprendizaje se obtuvo.

#	Error	Qué vi	Causa	Cómo lo resolví	Qué aprendí
1	`ModuleNotFoundError: No module named 'app'`	Falló al ejecutar tests/test_domain.py directamente con python.	Python solo busca módulos en la carpeta del archivo que se ejecuta (tests/), y app está un nivel más arriba.	Ejecuté desde la raíz del proyecto con python -m pytest y agregué un pytest.ini con pythonpath = .	El lugar desde donde se ejecuta un programa importa tanto como el código. Los archivos de pruebas se corren con pytest, no con python archivo.py.
2	`TypeError: DomainEvent() takes no arguments`	Apareció al llamar a confirm() por primera vez.	La clase DomainEvent estaba definida sin campos, así que Python no aceptaba "OrderConfirmed" ni self.id.	La definí como @dataclass(frozen=True) con name, order_id y occurred_at.	Hay que leer el mensaje de error completo, porque dice qué falla. Un Domain Event debe ser inmutable: registra un hecho que ya ocurrió.
3	`KeyError: 'sku'`	Falló el escenario E08 al buscar el precio de SKU003.	productos.csv empieza con un carácter invisible (BOM) que Excel agrega al guardar. Con encoding="utf-8", Python leía el encabezado como \ufeffsku.	Abrí el archivo con encoding="utf-8-sig" y reutilicé la función rows() de cargar_csv.py, que ya lo manejaba.	Un dato puede verse bien y estar mal por dentro. Conviene reutilizar el código que ya funciona y no suponer cómo están nombradas las cosas sin comprobarlo.

8. Preguntas

¿Por qué se usa Decimal en lugar de float para precios?

Un float guarda los números en binario y no representa con exactitud muchos decimales: en Python, 0.1 + 0.2 da 0.30000000000000004. En dinero esos errores se acumulan y pueden alterar comparaciones como total > 500. Decimalhace aritmética decimal exacta. En el proyecto, Money usa Decimal, y por eso O010 vale exactamente 500.00 y se puede probar que 500 > 500 es falso.

¿Cuál es la diferencia entre contar renglones y sumar unidades?

Un renglón es una línea del pedido (un producto), y las unidades son la cantidad pedida de ese producto. La regla limita las unidades. O009 tiene 1 renglón con 19 unidades: al agregar 2 más, contar renglones aceptaría el pedido, pero sumar unidades da 21, supera el máximo de 20 y add_item() lo rechaza.

¿Qué ventaja tiene encapsular una regla en Specification?

La regla se vuelve un objeto con nombre que se prueba por separado, se reutiliza y se combina con &, | y ~. Así no se repiten expresiones booleanas largas en varios lugares. La regla del 10 % se define una sola vez, y si cambia un umbral, se edita una línea.

¿Por qué los eventos se registran solo después de una transición válida?

Un Domain Event comunica que un hecho ya ocurrió. Si se registrara antes de validar, una operación rechazada anunciaría algo que nunca pasó. Por eso cada método valida, cambia el estado y registra el evento al final. En E12, el segundo confirm() falla antes de esa línea y los eventos siguen en uno solo.

¿Qué parte del código podrías reutilizar si reemplazas FastAPI por otra interfaz?

Todo excepto app/api.py: domain.py, specifications.py, cargar_csv.py y las pruebas de dominio. El núcleo no depende de FastAPI, como lo muestra escenarios_run.py, que ejecuta las mismas reglas sin servidor. Solo habría que escribir un controlador nuevo que llame a los métodos de Order y traduzca sus excepciones a la respuesta de esa interfaz.

9. Repositorio Github

https://github.com/yahirvela19/Unidad4